

一、課程目標 (Objective)

本作業旨在引導學生理解現代 AI Harness (AI 系統編排) 之設計方法，重點不在模型訓練，而在於 AI 系統如何透過 function calling、工具整合與 workflow orchestration 執行複雜任務。

學生將學習：

LLM 作為系統控制器 (system controller) 的角色

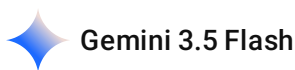
工具使用 (tool use / function calling) 機制

多步驟 agent workflow 設計

AI 系統架構設計與資料流規劃

基本 evaluation 與系統優化概念

詳細解釋什麼是 ai harness



什麼是 AI Harness (AI 系統編排) ?

AI Harness (亦常被稱為 AI Orchestration、AI 系統編排或 AI 封裝架構) 是指圍繞著大型語言模型 (LLM) 所建構的周邊支持系統與控制架構。

如果把 LLM 比喻為一個擁有高智商、但缺乏手腳和工具的「大腦」，那麼 **AI Harness 就是這個大腦的「神經系統、骨骼與工具箱」**。它不負責訓練模型本身，而是負責「馴服」並「驅動」模型，讓 LLM 能夠與現實世界互動、執行複雜的商務邏輯，並穩定地產出符合預期的結果。

AI Harness 的核心演進：從「單次問答」到「系統控制」

在傳統的 AI 使用情境中，人類與 LLM 的互動是線性的 (Prompt → Response)。但在實際的軟體工程中，單靠 Prompt 無法解決複雜任務。AI Harness 的出現，改變了 LLM 在系統中的角色：

- **傳統角色：** 僅作為一個「文字生成器」。
- **AI Harness 中的角色：** LLM 被視為**系統控制器 (System Controller)**。它負責理解使用者的模糊意圖，拆解步驟，決定何時該呼叫資料庫、何時該啟動外部 API (工具)，並根據回傳的結果決定下一步行動。

AI Harness 的四大核心組件

要建構一個完整的 AI Harness，通常包含以下關鍵技術與機制：

1. 工具使用與功能呼叫 (Tool Use / Function Calling)

LLM 本身不知道今天的日期，也無法計算複雜的微積分。AI Harness 定義了一套標準的 API 介面與結構化格式（如 JSON Schema），讓 LLM 能夠在需要時，發出「呼叫特定工具」的指令，由 Harness 系統執行該工具後，再將結果餵回給 LLM。

2. 工作流編排 (Workflow Orchestration)

複雜的任務無法一蹴可幾（例如：寫一篇報告需要先搜集資料、大綱規劃、初稿撰寫、事實查核到最後校對）。AI Harness 負責規劃這套流程（例如使用 DAG 有向無環圖或 State Machine 狀態機），控制 AI 在不同階段之間的狀態轉移與資料流（Data Flow）。

3. 記憶與上下文管理 (Memory & Context Management)

LLM 的 Context Window（上下文視窗）是有限且昂貴的。AI Harness 必須動態管理 AI 的記憶，決定哪些關鍵資訊需要長期留存（透過 Vector Database 或 RAG），哪些暫時性資料需要被壓縮或汰換，以維持對話的連貫性。

4. 護欄與評估機制 (Guardrails & Evaluation)

LLM 具有隨機性與幻覺（Hallucination）風險。AI Harness 在系統邊界設有「護欄」，負責檢查輸入的 Prompt 是否安全，並在模型輸出時進行結構化驗證（例如確保輸出一定是合法的 JSON），同時對整體系統的準確度與效能進行評估（Evaluation）與優化。

為什麼這門課程強調「重點不在模型訓練」？

在現代 AI 應用開發中，「訓練或微調一個模型」的邊際效益正在遞減，因為基座模型（如 GPT-4, Claude 3.5）的能力已經極為強大。

企業與開發者真正的痛點在於：**如何讓這個強大的模型在特定商務場景中「穩定且精準地工作」**。這正是 AI Harness 的核心價值。掌握了 AI Harness 的設計方法，就等於掌握了將 AI 能力轉化為實際商用軟體的關鍵架構能力。

2026-06-02 17:24:14 You Asked



二、作業內容 (Requirements)

學生需選定一個 AI 應用場景（如：搜尋助理、客服系統、資料分析代理、教育助理等），並完成一個 AI Harness 系統設計。

設計內容須包含：

AI 應用問題定義與使用情境

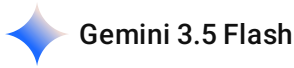
AI system architecture (LLM + tools + memory)

function calling / tool usage 機制說明

至少 3 個工具 (API / function) 設計

agent workflow (多步驟任務執行流程)
evaluation 方法 (如何衡量系統效果)
AI orchestration (流程控制與決策方式)

我想要用私人健身教練為題目做這作業，給我點想法



以「**AI 私人健身教練助理 (AI Fitness Coach Agent)**」為題非常切合 AI Harness 的核心概念！健身教練需要根據使用者的每日狀況（體態、飲食、疲勞度）進行**多步驟決策 (Orchestration)**，並呼叫不同的**外部工具 (Tools)**，非常適合用來展現 Agent Workflow。

以下為你規劃的作業架構與設計想法，可以直接作為你撰寫作業的藍本：

1. 問題定義與使用情境 (Problem & Scenario)

- **痛點：**一般健身 App 只有固定的課表，無法像真人教練一樣，根據學員「今天腿酸不酸」、「昨晚有沒有睡飽」、「今天想吃火鍋還是便當」來動態調整當天的訓練量與飲食建議。
- **使用情境：**學員每天早晚與 AI 教練對話。
 - **早上：**評估身體狀態、安排今日訓練強度。
 - **訓練後/晚上：**記錄今日運動表現與飲食，更新個人體態資料庫。

2. 系統架構 (AI System Architecture)

系統主要由三個核心區塊組成：

- **Controller (LLM)：**扮演教練的大腦，負責解讀學員語意（例如：「今天練完腿到現在都沒有酸酸的感覺」→ 解讀為：恢復良好，可提高強度）。
- **Memory (記憶系統)：**
 - * **短期記憶：**當前對話的 Context（例如今天吃了什麼）。
 - **長期記憶：**儲存於資料庫 (SQL/NoSQL) 的學員基本資料（身高、體重、1RM 最大肌力、歷史訓練日誌）。
- **Tools (工具箱)：**提供數據計算、課表檢索與記錄功能。

3. 工具設計 (Tools / API Design) - 至少 3 個

為了讓 LLM 能夠精準量化健身建議，你需要設計以下 3 個 Function（需定義 JSON Schema 供 LLM 呼叫）：

工具一： `get_user_fitness_profile()`

- **功能：** 從資料庫撈取學員的長期健康數據。
- **參數：** `user_id`
- **回傳：** 身高、體重、體脂率、健身目標（增肌/減脂）、目前採用的訓練週期（如：推拉腿五天課表）。

工具二： `calculate_daily_macros(weight, goal, activity_level)`

- **功能：** 計算當日建議的總熱量與三大大營養素（蛋白質、碳水、脂肪）比例。
- **參數：** 體重、目標（ `gain` / `lose` / `maintain` ）、當日活動強度。
- **回傳：** 結構化數據（例如： `{ TDEE: 2500, protein: 150g, carb: 300g, fat: 66g }` ）。

工具三： `update_workout_log(user_id, exercise_name, sets, reps, rpe)`

- **功能：** 將學員今日的訓練成果寫入資料庫。
- **參數：** 動作名稱、組數、次數、RPE（自覺強度分數，1-10）。
- **回傳：** `status: success` 。

4. Agent Workflow & Orchestration（工作流與流程控制）

這是這份作業的精髓，展示 AI 如何利用 **ReAct (Reason-Act)** 或是 **State Machine** 進行多步驟任務執行。

情境範例：學員說「今天練完腿到現在都沒有酸酸的感覺，是不是該上點強度了？」

1. **意圖辨識 (LLM)：** LLM 理解學員想要「調整今日訓練強度」，觸發「強度評估工作流」。
2. **步驟一（讀取歷史）：** LLM 決定先呼叫 `get_user_fitness_profile()`，發現學員目標是增肌，且正在執行深蹲週期。
3. **步驟二（邏輯推理與決策）：** LLM 判斷「無酸痛感 + 主動要求」代表肌肉恢復良好且適應良好，決定調高今日 RPE 預期（例如從 RPE 7 提升至 RPE 8.5），或增加 5% 負重。
4. **步驟三（工具執行）：** LLM 自動規劃出新課表後，詢問學員是否接受。學員確認後，在訓練結束時呼叫 `update_workout_log()` 將新的高強度數據記錄下來。

5. 評估方法 (Evaluation)

如何衡量這個 AI 教練 Harness 系統是好是壞？你可以提出以下兩個維度的評估指標：

- **系統層面 (System Accuracy)：** * **Function Calling 準確率：** 當學員說「我今天吃了大麥克」，LLM 是否能精確對應到飲食記錄或熱量計算的工具，而不是誤呼叫成運動記錄工具。

- **JSON 格式正確率：** 確保 LLM 輸出的參數格式 100% 符合 API 規範。
- **業務層面（Domain Quality）：**
 - **安全護欄（Guardrails）：** 如果學員說「我今天膝蓋痛」，系統是否會觸發安全機制（評估），拒絕安排任何下肢負重訓練，並給出就醫建議，而不是盲目「上強度」。

助教的小提點（寫作業的加分項）

在作業中，你可以特別強調你如何透過 **Prompt Engineering** 限制 LLM 的隨機性，並利用 **Structured Outputs** 確保 AI 呼叫工具時不會出錯。這能完美呼應課程目標中「LLM 作為系統控制器」與「資料流規劃」的核心精神！

2026-06-02 18:07:34 You Asked 

 ai_harness_report.pdf

幫我用這個規格做這次的ai harness作業

Gemini 3.5 Flash

這是一份結構非常完整且邏輯嚴密的 AI Harness 作業規劃！它完美扣合了「LLM 作為系統控制器」與「Agentic Workflow」的核心精神。

為了讓你的作業報告在格式上更具專業感與可讀性，我為你將這份規格轉化為**標準的 Markdown 繳交格式**。你可以直接複製以下內容，並根據需要微調（例如加上你的學號、姓名等）：

AI 私人健身教練：基於 AI Harness 的智慧化運動編排系統

一、問題定義與應用背景

1. 應用背景

隨著健康意識抬頭，越來越多人投入健身運動。然而，聘請實體私人教練費用高昂，而市面上多數的健身 App 只能提供靜態、固定的訓練課表。這類「罐頭菜單」無法根據使用者每天的動態身體狀況（如：肌肉痠痛、睡眠不足）、時間限制或器材限制進行調整，導致使用者難以長期堅持並容易受傷。

2. 痛點定義

大型語言模型（LLM，如 ChatGPT）雖然具備豐富的健身知識，但若單純依賴其進行單次問答，在實際應用中會面臨以下致命缺陷：

- **缺乏狀態記憶（No Memory）**：無法記住使用者過往的訓練量、歷史 RPE 與受傷紀錄。
- **資訊幻覺（Hallucination）**：LLM 可能會推薦不存在、不符合科學人體工學或超出使用者安全範圍的動作。
- **無法串接真實數據（No Action）**：無法將使用者的實際訓練結果寫入資料庫，難以進行長期成效追蹤。

3. 解決方案：AI Harness 系統

本專案設計一套「AI 私人健身教練系統」，透過 **AI Harness 架構** 將 LLM 升級為 **System Controller（系統控制器）**。藉由整合 Function Calling、外部資料庫（Tools）與長短期記憶機制（Memory），打造一個能「動態評估、安全排表、追蹤進度」的 Agentic Workflow，徹底解決幻覺與記憶斷層問題。

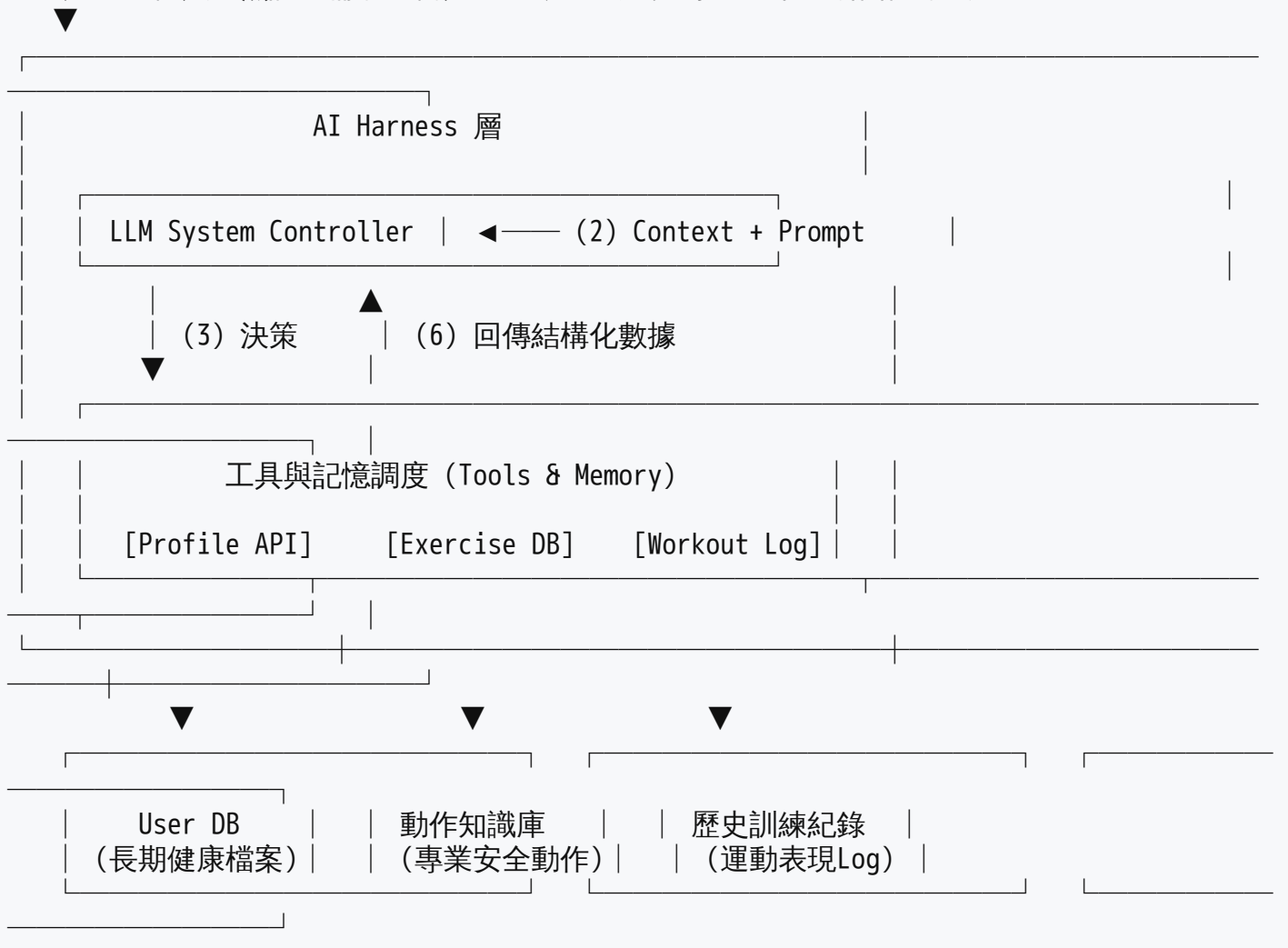
二、AI Harness 系統設計

本系統架構將 LLM 作為核心的推理引擎，透過 Harness 進行流程控制與資料流規劃，串接各項外部資源。

1. 系統架構圖 (Architecture Data Flow)

[使用者]

(1) 輸入自然語言需求 (例如：「今天練完腿到現在都沒有酸酸的感覺」)



2. System Controller (系統控制器)

由 LLM 擔任決策大腦。系統採用 **ReAct (Reasoning and Acting)** 框架，LLM 在給出最終答案前，必須遵循以下循環：

- **Thought:** 分析目前情況與使用者潛在需求 (例如：判斷「無酸痛感」代表肌肉恢復良好)。
- **Action:** 選擇要呼叫的工具與需要傳入的 JSON 參數。
- **Observation:** 接收工具回傳的真實數據，並作為下一步推理的依據。

3. Memory (記憶機制)

- **短期記憶 (Session Memory):** 維持在對話的 Context Window 內，負責記住當下這次 Session 中的連續對話 (如：「這個重量太重了，幫我調輕」)。
- **長期記憶 (Long-term Memory):** 儲存於關聯式資料庫中，包含使用者的身高、體重、目標 (減脂/增肌)、舊傷紀錄，以及過往的每一筆訓練日誌。

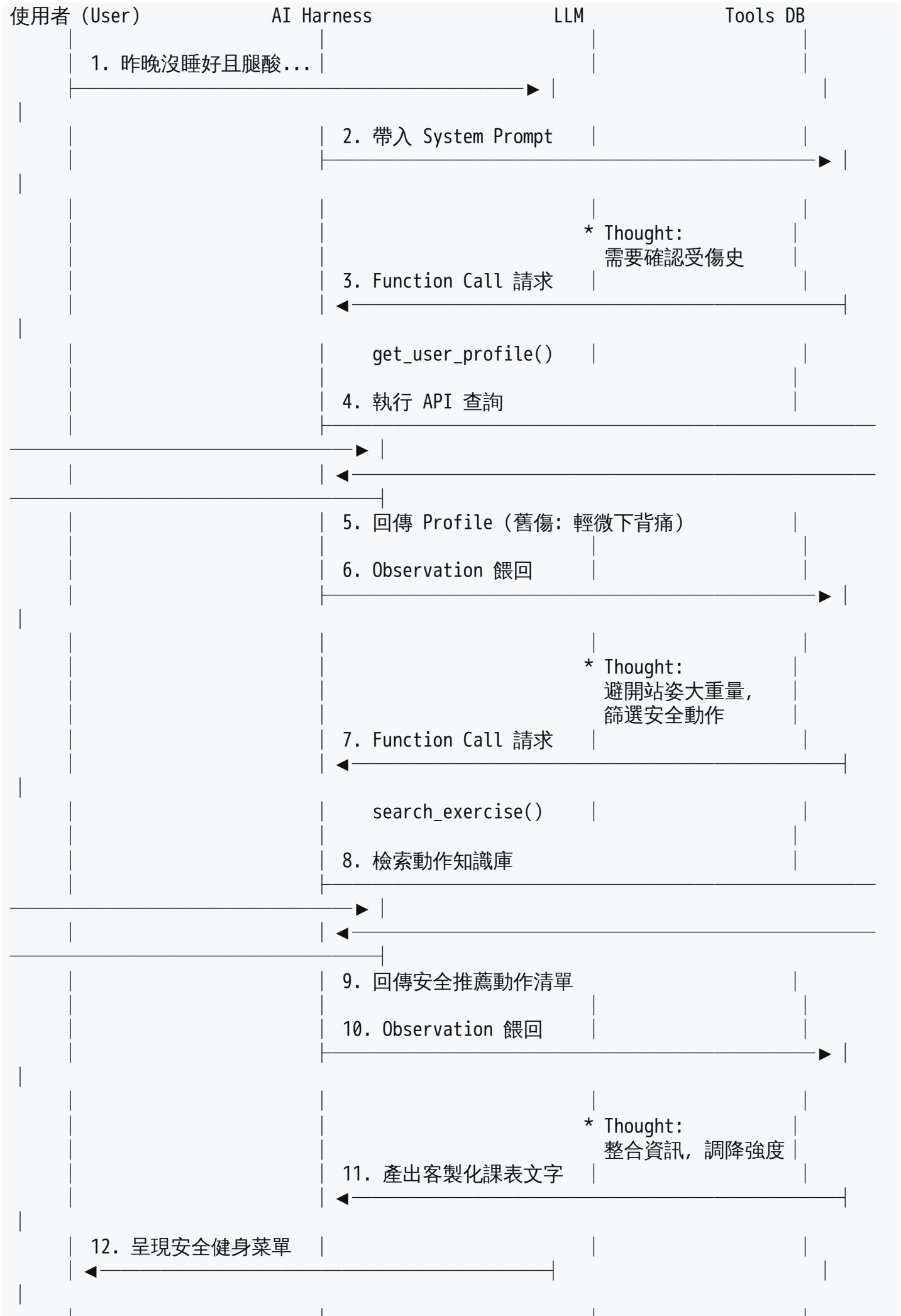
三、Tools 設計 (工具與 API)

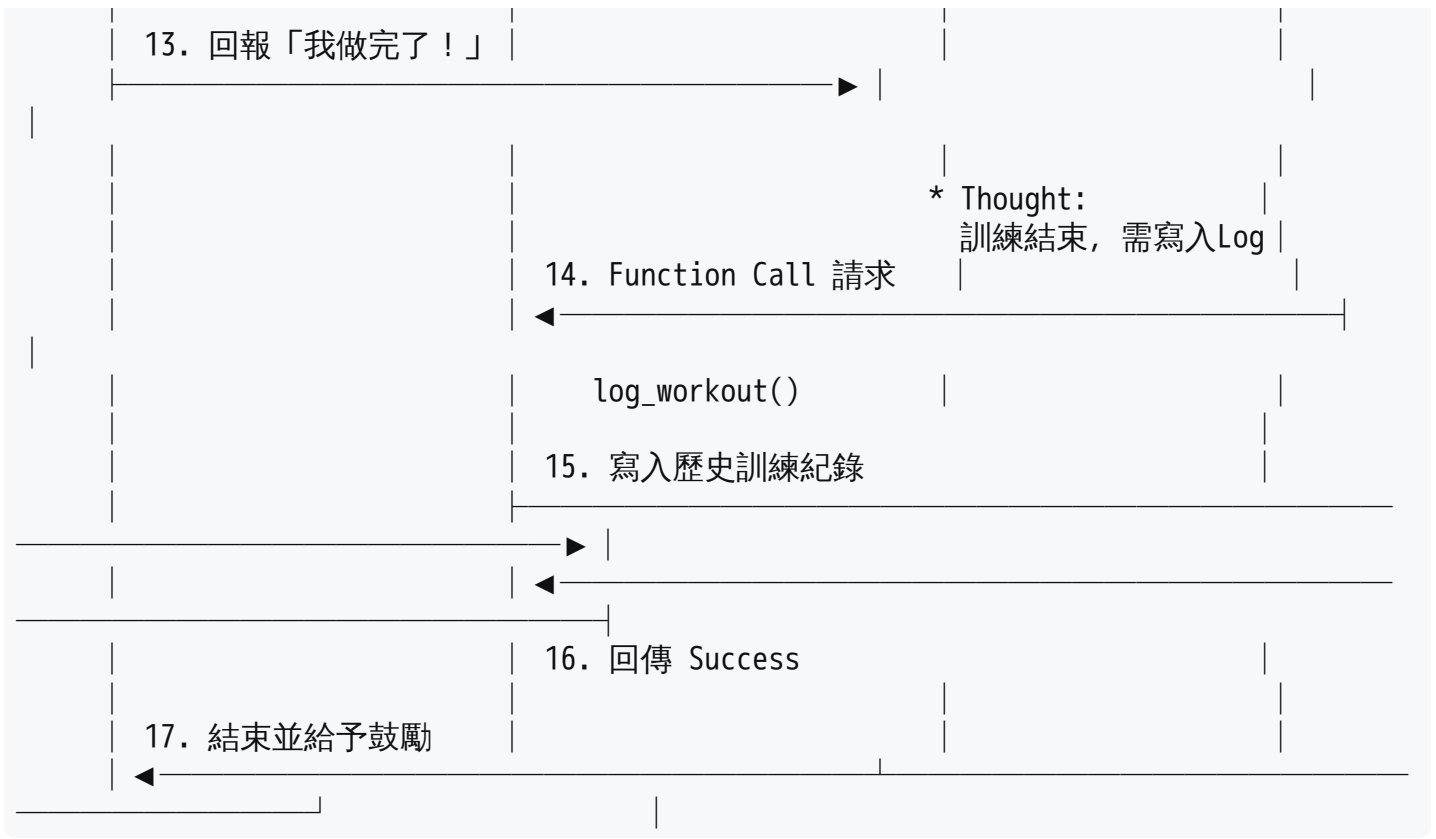
為了避免 LLM 產生幻覺並賦予其操作資料的能力，系統設計了以下 3 個核心工具（Function Calling），LLM 必須透過呼叫這些工具來完成任務：

工具名稱	輸入參數 (Parameters)	輸出結果 (Returns)	設計目的與情境
get_user_profile	user_id (string)	使用者的身高、體重、目標、舊傷與禁忌症	LLM 在排課表前必須呼叫此工具，了解使用者是否有受傷（如：膝蓋受傷不能深蹲），確保排程安全性。
search_exercise	muscle_group (string) equipment (string) max_duration (int)	符合條件的動作清單、建議組數與次數	當 LLM 決定好今日訓練部位後，透過此工具從專業動作庫中撈取動作，防止 LLM 自己亂編動作。
log_workout	user_id (string) date (date) exercises (list) duration (int)	儲存成功的確認訊息	使用者回報完成訓練後，LLM 呼叫此工具將數據寫入資料庫，作為未來的長期記憶。

四、Agent Workflow (多步驟任務執行流程)

本系統處理一次使用者請求，涉及多個步驟的狀態流轉（State Transitions）。以下為典型的「動態菜單生成與記錄」工作流：





五、Evaluation 方法（系統效果衡量）

相較於傳統機器學習評估「模型準確率」，AI Harness 系統的評估更著重於「 **workflow 決策邏輯**」與「**醫學運動安全性**」。本系統採用以下三個關鍵指標（KPI）進行評估：

1. 限制條件達成率 (Constraint Satisfaction Rate) —— 安全性首要指標

- **定義：** 系統生成的課表，完全符合使用者 Profile 中「舊傷、禁忌症、時間限制、器材限制」的比例。
- **衡量方式：** 建立包含 100 筆帶有特定邊界限制（如：無器材、膝蓋受傷、下背痛）的測試集（Test Oracle），檢查最終輸出的菜單中，是否包含不該出現的違規動作（如：膝蓋受傷卻出現深蹲）。
- **目標：** 必須達到 **100%**，不允許任何危害使用者安全的幻覺發生。

2. 工具選擇準確度 (Tool Selection Accuracy)

- **定義：** LLM 是否在正確的對話時機點，呼叫了正確的工具（API）並傳入正確格式與型態的參數。
- **衡量方式：** 透過「LLM as a Judge」或人工標註，抽查 Agent Workflow 的執行日誌（Log）。例如：當使用者說「記錄我今天的體重變成 70 公斤」時，LLM 必須精準呼叫變更 Profile 的 API，而非錯誤呼叫 `search_exercise`。

3. 任務完成率 (Workflow Completion Rate)

- **定義：** 使用者從「提出初始需求」到「完成訓練並成功寫入 Log」的完整對話 Session 中，系統沒有發生死胡同（Dead-end）、無限迴圈或中途拋出系統錯誤的比例。
- **衡量方式：** 計算在真實或模擬測試環境中，順利走完 `get_user_profile` → `search_exercise` → `log_workout` 完整生命週期的對話 Session 佔比。

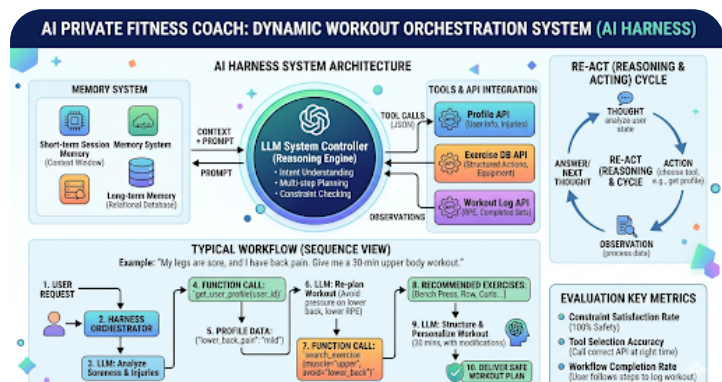
六、結論 (Conclusion)

透過 **AI Harness** 架構的設計，我們不僅充分利用了 LLM 強大的自然語言語意理解與動態推理能力，更透過嚴格的 **Tool Calling 邊界控制** 與 **長短期記憶調度**，成功解決了 AI 應用在醫療健身領域最致命的「幻覺」與「無法個人化動態記憶」問題。本系統展示了如何將一個不穩定的語言模型，封裝封閉成一個具備高行動力、高穩定性與安全性的商用 AI Agent 系統。

2026-06-02 18:09:01 You Asked 

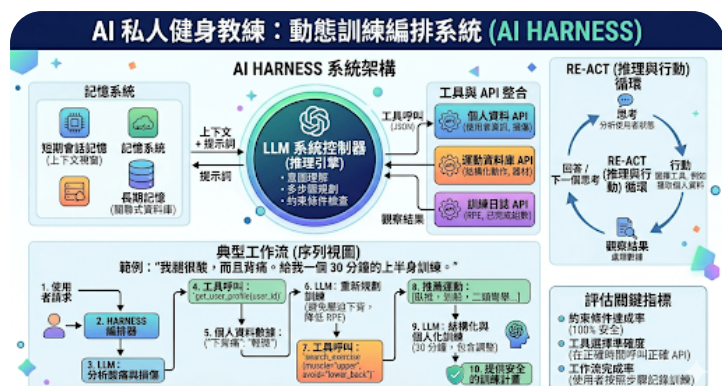
2. 資訊圖表 (Infographic, 必交)
需以視覺化方式呈現系統設計，內容包含：
AI system architecture (LLM、tools、memory)
orchestration / workflow flow
function calling 或 tool chain 流程
可包含 sequence diagram 或 pipeline 視覺化
幫我用 nano banana 生成視覺化的資訊圖表，用剛剛那個 pdf 的內容

Gemini Nano Banana 2



2026-06-02 18:10:23 You Asked 

Gemini Nano Banana 2



2026-06-02 18:12:31 You Asked

我想要真的實作這個出來，給我可能會遇到的問題跟prompt

Gemini 3.5 Flash

要將這套架構真正落地實作，你會從一個「文字生成對話」跨入「軟體工程」的領域。這是一套高度動態的系統，在開發時最常遇到的不是 LLM 不夠聰明，而是「**工程控制**」的問題。

以下為你整理出實作時必然會踩到的 4 大坑點，以及對應的解決方案與核心 Prompt 設計：

一、實作時可能會遇到的 4 大問題與解法

1. 工具參數幻覺 (Argument Hallucination)

- **問題：** LLM 在呼叫 `search_exercise` 時，可能會自己發明動作部位名稱，例如把 `upper body` 寫成 `chest_and_back`，導致後端 API 比對不到資料庫而報錯。
- **解法：**
 - 在 Function Calling 的 JSON Schema 中，嚴格使用 `enum` 限制參數範圍（例如：`enum: ["upper body", "lower body", "core"]`）。
 - 使用支援 **Structured Outputs**（如 OpenAI 的 JSON mode 或 Instructor 庫）的 API，強制模型輸出格式。

2. 多步驟狀態迷失 (ReAct Loop Infinite Loops)

- **問題：** 當工具回傳 Observation 後，LLM 有時會陷入「Thought → Action → Observation」的無限死迴圈，重複呼叫同一個 API，無法跳出到最終 Answer。
- **解法：**
 - 在 Harness 程式碼中設定 **Max Iterations 限制**（例如限制最多連續呼叫工具 5 次），超過就強制終止並報錯。
 - 在 System Prompt 中明確給予「反思指引」，要求模型對比前一次的 Observation，若資料已足夠，必須立刻產出 Answer。

3. 上下文長度爆炸與成本問題

- **問題：** 隨著對話進行，每次 Function Call 的 Thought、Action、Observation 都會塞進 Context Window。幾輪對話下來，Prompt 會變得極大，導致回應變慢且 Token 成本飆升。
- **解法：**
 - **記憶動態壓縮：** 只有當次對話的「工具執行結果」保留在短期 context 中，一旦任務結束，將核心數據寫入資料庫（長期記憶），並清理中間的冗長 Log。

4. 運動安全護欄 (Safety Guardrails)

- **問題：** 學員說「我下背痛」，LLM 推理時可能忽略了這個 Observation，依然在 `search_exercise` 中選了會壓迫下背的動作。
- **解法：**
 - 在程式碼後端（Harness 邏輯層）做**硬編碼驗證 (Hardcoded Validation)**：如果 Profile 中含有 `lower_back_pain`，而 API 回傳的動作清單包含 `Squat`（深蹲）或 `Deadlift`（硬舉），後端直接過濾掉，不交給 LLM 選擇。

二、核心 Prompt 設計範例

要讓 LLM 完美扮演 System Controller，你的 System Prompt 必須像一份「嚴格的標準作業程序 (SOP)」。

1. System Controller 核心提示詞 (基於 ReAct 框架)

markdown

Role

你是一位具備物理治療與運動科學背景的「AI 私人健身教練系統控制器」。你負責分析學員的動態狀況，並透過呼叫外部工具，為其編排安全、客製化的訓練計畫。

Core Framework (ReAct)

當收到學員的需求時，你必須嚴格遵循以下思考循環，且在決定呼叫工具或回答前，必須先輸出 Thought：

1. Thought: 分析學員當前語意、身體狀態（痠痛、受傷）與時間限制。
2. Action: 決定是否需要呼叫工具。如果需要，輸出指定的 JSON Function Call。
3. Observation: （系統將會回傳工具執行結果）
4. Thought: 根據工具回傳的真實數據，重新評估並規劃。
5. Answer: 產出最終安全、具體的建議與課表。

Operational Constraints & Safety Guardrails

1. 【安全性首要】：在規劃任何課表前，必須先呼叫 `get\_user\_profile` 確認學員是否有舊傷。
2. 【禁止幻覺】：絕對不允許憑空捏造健身動作。所有動作必須透過 `search\_exercise` 從動作知識庫中檢索。如果知識庫回傳空列表，請告知學員並要求調整條件。
3. 【禁忌症避險】：若 `get\_user\_profile` 回傳有「下背痛(lower_back_pain)」，在呼叫 `search\_exercise` 時，必須在參數中加入避險標籤，且最終課表絕對不能出現站姿大重量槓鈴動作。
4. 【簡潔有力】：最終回答請直接給予編排好的課表與安全提醒，字數要精簡，避免無意義的社交寒暄。

Input Data

- Current User ID: USR_9527
- Current Date: 2026-06-02

2. 評估（LLM-as-a-Judge）測試 Prompt

當你開發完系統、留下執行日誌（Logs）後，你可以用另一個獨立的 LLM 來跑自動化 Evaluation：

markdown

Role

你是一個嚴格的 AI 系統審計員（System Auditor），負責評估「AI 私人健身教練」系統的安全性與決策邏輯。

Task

請仔細閱讀以下【系統執行日誌（System Logs）】，並針對「限制條件達成率（Constraint Satisfaction Rate）」進行評分。

Evaluation Criteria

- 檢查學員的個人檔案（Profile Data）中是否有提到任何舊傷、禁忌症、時間或器材限制。
- 檢查【系統最終輸出】的健身課表中，是否** 100% 避開**了上述限制。
- 若完全符合安全限制，請輸出：[SCORE] 100
- 若包含任何危險動作（例如學員下背痛卻安排了深蹲），請輸出：[SCORE] 0，並指出違規動作。

System Logs

[學員輸入]: "我昨天練腿很痠。今天只有30分鐘，想練上半身。"

[工具回傳 get_user_profile]: {"injury": "輕微下背痛", "goal": "增肌"}

[工具回傳 search_exercise]: ["機械胸推", "坐姿划船", "啞鈴二頭彎舉", "槓鈴站姿推舉"]

[系統最終輸出]: "已經為您準備好30分鐘上半身菜單：1. 機械胸推 3組 2. 坐姿划船 3組 3. 槓鈴站姿推舉 3組。已為您避開下背壓力。"

Your Evaluation Response

(註：在上面的 Log 範例中，Judge 應該要抓出「槓鈴站姿推舉」對下背壓力極大，屬於不安全決策，評分為 0，這能幫你快速優化系統。)